

PROGRAM 1: Exploring Pre-trained Word Vectors and Word Relationships

```
!pip install gensim
```

```
from gensim.downloader import load
```

```
model = load("glove-wiki-gigaword-50")
```

```
print(model.most_similar(positive=["king", "woman"], negative=["man"], topn=1))
```

```
print(model.most_similar(positive=["paris", "italy"], negative=["france"], topn=1))
```

```
print(model.most_similar("programming", topn=5))
```

Program 2

```
!pip install gensim matplotlib scikit-learn
```

```
from gensim.downloader import load
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.decomposition import PCA
```

```
model = load("glove-wiki-gigaword-50")
```

```
print("Model ready")
```

```
words = [
```

```
    "computer", "software", "hardware", "programming",
```

```
    "internet", "data", "network", "java",
```

```
    "coding", "database", "king", "queen"
```

```
]
```

```
vectors = [model[w] for w in words]
```

```
pca = PCA(n_components=2)
```

```
points = pca.fit_transform(vectors)
```

```
for i in range(12):
    plt.scatter(points[i][0], points[i][1])
    plt.text(points[i][0], points[i][1], words[i])

plt.show()

input_word = input("Enter a word: ")

# Get the most similar words (top 5)
similar_words = model.most_similar(input_word, topn=5)
print(similar_words)
```

Program 3

```
!pip install gensim matplotlib
from gensim.models import Word2Vec

# Step 1: Medical dataset
medical_data = [
    ["patient", "doctor", "nurse", "hospital", "treatment"],
    ["cancer", "chemotherapy", "radiation", "surgery", "recovery"],
    ["infection", "antibiotics", "diagnosis", "disease", "virus"],
    ["heart", "disease", "surgery", "cardiology", "recovery"]
]

# Repeat data so model learns better
medical_data = medical_data * 200

# Step 2: Train Word2Vec
```

```
model = Word2Vec(  
    sentences=medical_data,  
    vector_size=20,  
    window=2,  
    min_count=1,  
    workers=1,  
    epochs=200  
)
```

```
print("Model training complete!\n")
```

```
# Step 3: Function to check semantic similarity
```

```
def check_word(word):
```

```
    if word in model.wv:
```

```
        print("Words related to:", word)
```

```
        for w, sim in model.wv.most_similar(word, topn=3):
```

```
            print(w, "->", round(sim, 2))
```

```
        print()
```

```
    else:
```

```
        print(word, "not in vocabulary\n")
```

```
# Step 4: Test multiple medical terms
```

```
check_word("patient")
```

```
check_word("virus")
```

```
check_word("heart")
```

Program 4

dictionary of similar words (simulating word embeddings)

```
similar_dict = {  
    "explain": ["describe", "clarify"],  
    "machine": ["device", "system"],  
    "learning": ["training", "education"],  
    "ai": ["artificial intelligence", "automation"]  
}
```

function to find similar words

```
def find_similar_words(word):  
    return similar_dict.get(word.lower(), [])
```

function to enrich prompt

```
def enrich_prompt(prompt):  
    words = prompt.split()  
    enriched = []  
  
    for word in words:  
        sim = find_similar_words(word)  
  
        if sim:  
            new_word = word + " (" + ", ".join(sim) + ")"  
            enriched.append(new_word)  
        else:  
            enriched.append(word)
```

```

        return " ".join(enriched)

# original prompt
prompt = "Explain machine learning"

# enriched prompt
enriched_prompt = enrich_prompt(prompt)

print("Original Prompt:", prompt)
print("Enriched Prompt:", enriched_prompt)

print("\nResponse for Original Prompt:")
print("Machine learning allows computers to learn from data.")

print("\nResponse for Enriched Prompt:")
print("Machine learning is a field of AI where systems learn patterns from data and improve automatically.")

```

Program 5

```

!pip install gensim

from gensim.downloader import load

import random

print('Loading pre-trained Glove Model (50 Dimensions)...')

model = load('glove-wiki-gigaword-50')

print('model loaded successfully')

def create_paragraph(iw, sws):
    random.shuffle(sws)

```

```
words_str = ', '.join(sws)

return f'The topic of {iw} is fascinating, often linked to terms like: {words_str}.'

seed_word='hacking'

similar_words= model.most_similar(seed_word, topn=5)

words = [word for word, _ in similar_words]

paragraph = create_paragraph(seed_word, words)

print(paragraph)
```

Program 6

```
# Step 1: Install and import the library

# Run this in Google Colab if transformers is not installed

!pip install transformers

from transformers import pipeline
```

```
# Step 2: Load the sentiment analysis pipeline

sentiment_analyzer = pipeline("sentiment-analysis")
```

```
# Step 3: Define sentences

sentences = [

    "I love using this product! It makes my life so much easier.",

    "The service was terrible, and I'm very disappointed.",

    "It's an average experience, nothing special but not bad either.",

    "I wish I had made this purchase, worst product",

    "Received damaged product, i want a replacement"

]
```

```
# Step 4: Analyze sentiment
```

for sentence in sentences:

```
result = sentiment_analyzer(sentence)[0]
```

```
print("Sentence:", sentence)
```

```
print("Sentiment:", result['label'])
```

```
print("Score:", round(result['score'], 2))
```

```
print()
```

Program 7

```
!pip install transformers
```

```
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
```

```
text = """India has made remarkable progress in space exploration through the efforts of the  
Indian Space Research Orga
```

```
One of its most notable achievements was the Chandrayaan missions, which aimed to  
explore the Moon and study
```

```
Chandrayaan-3 successfully landed near the Moon's south pole, making India the first  
country to achieve this m
```

```
India also launched the Mars Orbiter Mission, known as Mangalyaan, which made India the  
first Asian nation to
```

```
and the first country in the world to do so on its maiden attempt. ISRO has developed  
reliable launch vehicle
```

```
to deploy satellites for communication, weather forecasting, and navigation. These missions  
have enhanced Ind
```

```
globally and demonstrated its ability to conduct cost-effective space research. In the future,  
India plans to
```

```
its first human spaceflight mission, marking another major step in the nation's space  
journey."""
```

```
tokenizer = AutoTokenizer.from_pretrained("facebook/bart-large-cnn")
```

```
model = AutoModelForSeq2SeqLM.from_pretrained("facebook/bart-large-cnn")
```

```
inputs = tokenizer(text, max_length=1024, return_tensors="pt", truncation=True)
summary_ids = model.generate(inputs["input_ids"], num_beams=4, max_length=150)
print(tokenizer.batch_decode(summary_ids, skip_special_tokens=True))
```

Program 8

```
!pip install -q langchain langchain-cohere langchain-community cohere
import os
from google.colab import drive, userdata
from langchain_cohere import ChatCohere
from langchain_core.prompts import PromptTemplate
from langchain_community.document_loaders import TextLoader
os.environ["COHERE_API_KEY"] = "yXefXa79G6Y22LMHVoNADjJca7exoc8F8AQiivd7"
drive.mount('/content/drive')
file_path = '/content/drive/MyDrive/sample.txt'
loader = TextLoader(file_path)
document = loader.load()
content = document[0].page_content
template = """
Please provide a summary of the text and then list 3 key takeaways.

Document:
{document_text}

Format:
SUMMARY:
KEY TAKEAWAYS:
1.
2.
```

3.

```
"""
```

```
prompt = PromptTemplate.from_template(template)
```

```
llm = ChatCohere(model="command-a-03-2025")
```

```
formatted_prompt = prompt.format(document_text=content)
```

```
response = llm.invoke(formatted_prompt)
```

```
print(response.content)
```